JOBSHEET 16

SISTEM AUTENTIKASI LOGIN TERINTEGRASI DATABASE

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_16

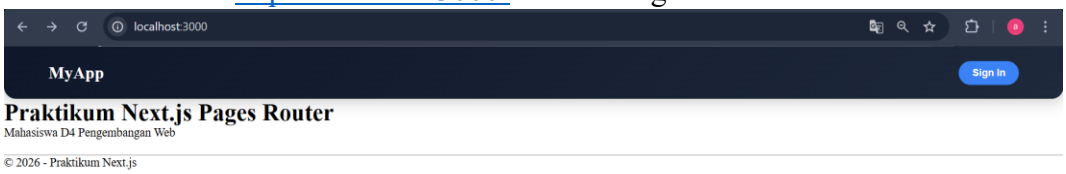
C. Langkah Praktikum

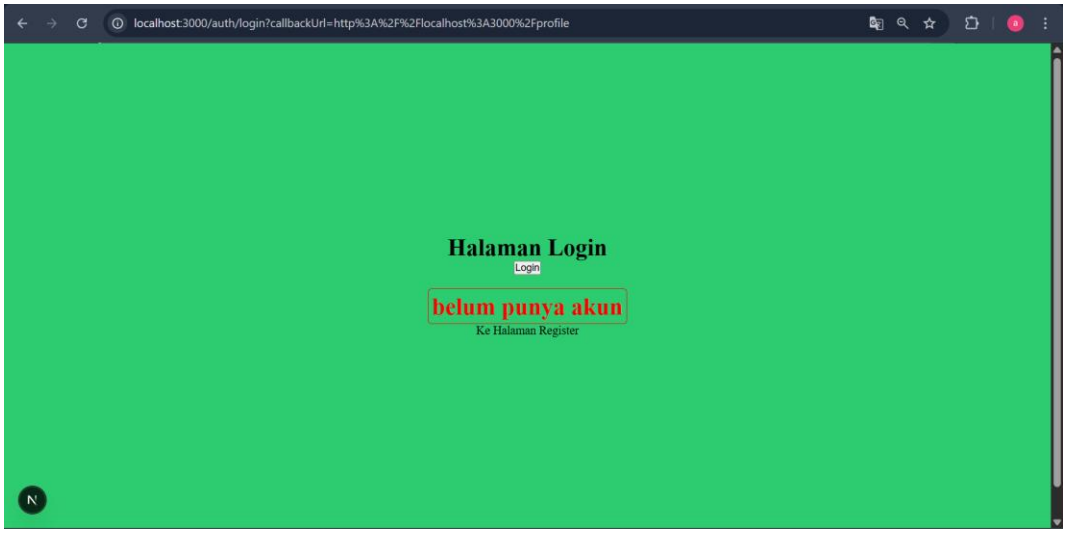
BAGIAN 1 – Custom Login Page

- Tambahkan custom page di NextAuth line 55-57
  Src/pages/api/auth/ [...nextauth].ts

```
1    // pengarah sign in ke halaman login
2    pages: {
3      signIn: "/auth/login",
4    },
5  };
6
7  export default NextAuth(authOptions);
8
```

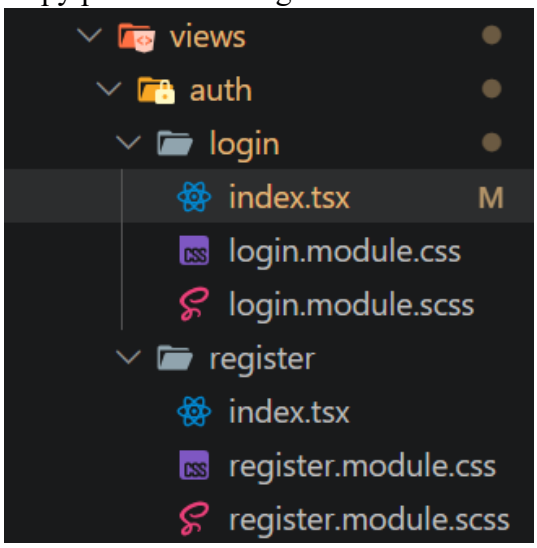
- Jalankan browser http://localhost:3000/ dan klik sign in maka akan diarahkan ke login





BAGIAN 2 – Handle Login di Frontend

- Copy paste isi dari register/index.tsx ke file login/index.tsx



- Copy paste isi dari register/register.module.scss ke file login/login.module.scss
- Semua text register pada file index.tsx pada folder login diubah menjadi login

```python
import Link from "next/link";
import style from "../../auth/login/login.module.scss";
import { useState } from "react";
import { useRouter } from "next/router";

const TampilanLogin = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push } = useRouter();
  const [error, setError] = useState("");

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    setError("");
    setIsLoading(true);
    event.preventDefault();
    const form = event.currentTarget;
    const formData = new FormData(event.currentTarget);
    const email = formData.get("email") as string;
    const fullname = formData.get("Fullname") as string;
    const password = formData.get("Password") as string;

    // validasi email dan password
    if (!email) {
      setError("Email wajib diisi");
      setIsLoading(false);
      return; // ← stop, tidak lanjut ke fetch
    }
    if (password.length < 6) {
      setError("Password minimal 6 karakter");
      setIsLoading(false);
      return; // ← stop, tidak lanjut ke fetch
    }

    const response = await fetch("/api/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, fullname, password }),
    });
    // const result = await response.json();
    // console.log(result);
    if (response.status === 200) {
      form.reset();
      // event.currentTarget.reset();
      setIsLoading(false);
      push("/auth/login");
    } else {
      setIsLoading(false);
      setError(
        response.status === 400 ? "Email already exists" : "An error occurred",
      );
    }
  };

  return (
    <div className={style.login}>
      {/* modifikasi line 41 */}
      {error && <p className={style.login__form__error}>{error}</p>}
      <h1 className={style.login__title}>Halaman login</h1>

      <div className={style.login__form}>

        <form onSubmit={handleSubmit}>

          {/* Email */}
          <div className={style.login__form__item}>
            <label
              htmlFor="email"
              className={style.login__form__item__label}
            >
              Email
            </label>
            <input
              type="email"
              id="email"
              name="email"
              placeholder="Email"
              className={style.login__form__item__input}
            />
          </div>

          {/* Fullname */}
          <div className={style.login__form__item}>
            <label
              htmlFor="Fullname"
              className={style.login__form__item__label}
            >
              Fullname
            </label>
            <input
              type="text"
              id="Fullname"
              name="Fullname"
              placeholder="Fullname"
              className={style.login__form__item__input}
            />
          </div>

          {/* Password */}
          <div className={style.login__form__item}>
            <label
              htmlFor="Password"
              className={style.login__form__item__label}
            >
              Password
            </label>
            <input
              type="password"
              id="Password"
              name="Password"
              placeholder="Password"
              className={style.login__form__item__input}
            />
          </div>

          {/* Button login */}
          <button
            type="submit"
            className={style.login__form__item__button}
            disabled={isLoading}
          >
            {isLoading ? "Loading..." : "login"}
          </button>

        </form>

        {/* Halaman login */}
        <br />
        <p className={style.login__form__item__text}>
          Sudah punya akun? <Link href="/auth/login">Ke Halaman login </Link>
        </p>
      </div>
      {/* Form selesai disini */}

    </div>
  );
};

export default TampilanLogin;
```

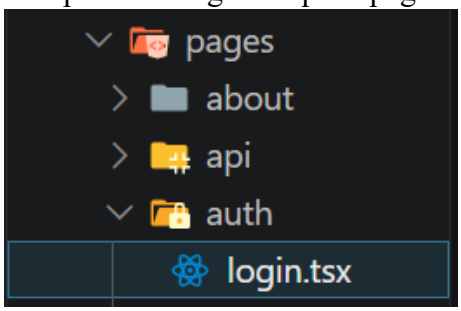
- Jangan lupa setting link hrefnya

```
{/* Halaman register */}
        <br />
        <p className={style.register__form__item__text}>
          Sudah punya akun? <Link href="/auth/register">Ke Halaman Register </Link>
        </p>
      </div>
```

- Lakukan hal yang sama pada file login.module.scss rubah text register menjadi login

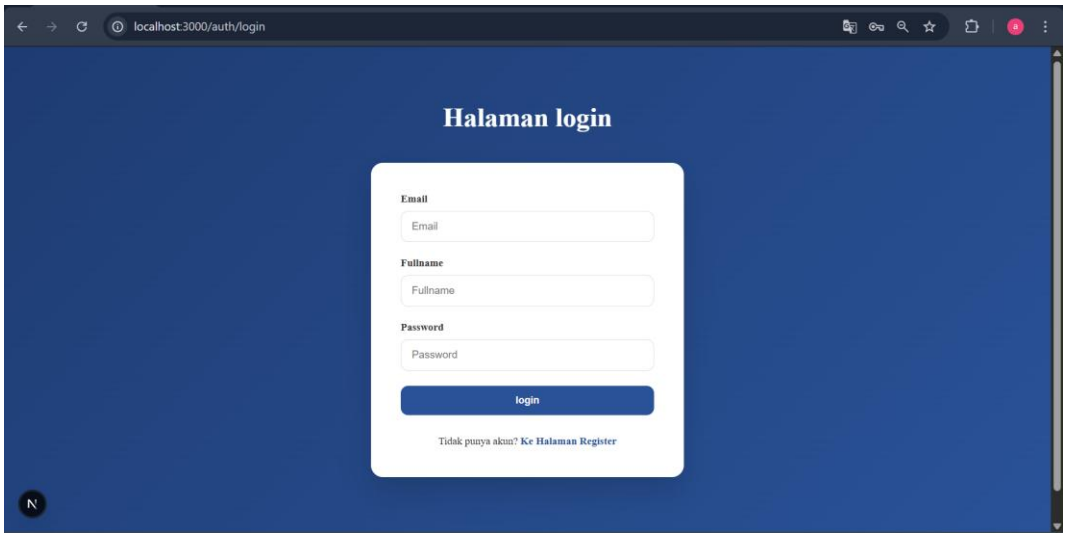
```scss
.login {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #1e3c72, #2a5298);
  padding: 20px;

  &__title {
    width: 100%;
    text-align: center;
    font-size: 36px;
    font-weight: 700;
    color: #ffffff;
    margin-bottom: 40px;
  }

  &__form {
    background: #ffffff;
    width: 100%;
    max-width: 420px;
    padding: 40px;
    border-radius: 16px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);

    &__item {
      display: flex;
      flex-direction: column;
      margin-bottom: 20px;

      &__label {
        font-size: 14px;
        font-weight: 600;
        margin-bottom: 8px;
        color: #333;
      }

      &__input {
        padding: 12px 14px;
        border-radius: 10px;
        border: 1px solid #ddd;
        font-size: 14px;
        transition: all 0.3s ease;

        &:focus {
          outline: none;
          border-color: #2a5298;
          box-shadow: 0 0 3px rgba(42, 82, 152, 0.2);
        }
      }

      &__button {
        width: 100%;
        padding: 12px;
        border-radius: 10px;
        border: none;
        background: #2a5298;
        color: #ffffff;
        font-weight: 600;
        cursor: pointer;
        transition: 0.3s ease;

        &:hover {
          background: #1e3c72;
          transform: translateY(-2px);
        }
      }

      &__text {
        font-size: 14px;
        color: #333;
        text-align: center;
        margin-top: 8px;

        a {
          color: #2a5298;
          font-weight: 600;
          text-decoration: none;

          &:hover {
            text-decoration: underline;
          }
        }
      }
    }

    &__error {
      color: red;
      font-size: 14px;
      margin-top: 8px;
    }
  }
}

/* Animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(15px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Responsive */
@media (max-width: 768px) {
  .login {
    &__form {
      padding: 30px 20px;
    }

    &__title {
      font-size: 26px;
    }
  }
}
```

- Cek pada file login.tsx pada pages/auth




```
1  import TampilanLogin from "@/views/auth/login";
2
3  const halamanLogin = () => {
4
5  return (
6      <>
7          <TampilanLogin />
8      </>
9      );
10 };
11
12 export default halamanLogin;
```

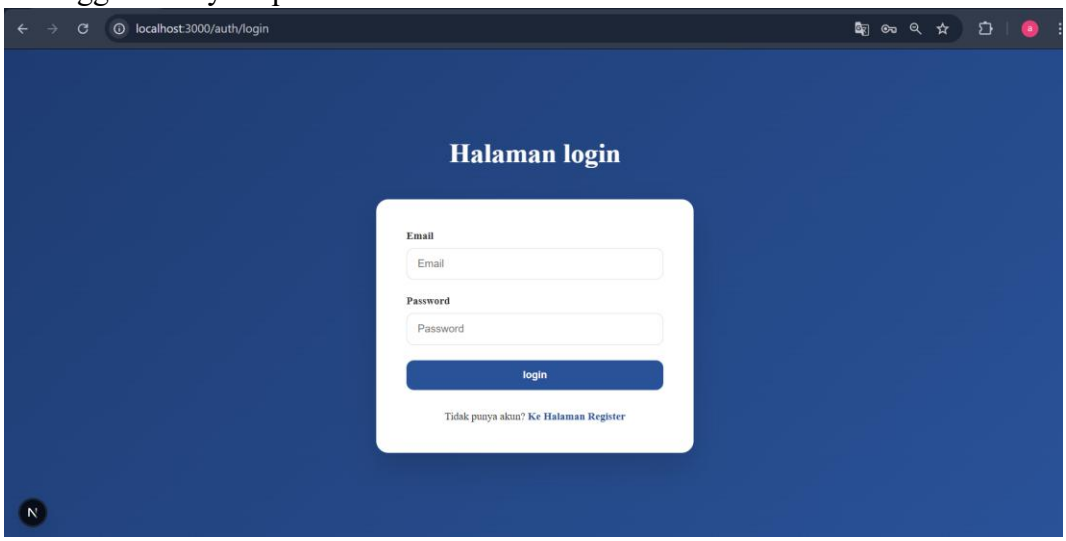
- Jalankan browser localhost:3000/auth/login. Tampilannya akan sama dengan register



- Pada tampilan login kita tidak perlu hapus fullname jadi pada folder views/auth/login/index.tsx hapus fullname

```
my-app-16 > src > views > auth > login > index.tsx > TampilanLogin
6   cons
77
78              className={style.login__form__item__input}
79            />
80          </div>
81
82          {/* Fullname */}
83          {/* <div className={style.login__form__item}>
84            <label
85              htmlFor="Fullname"
86              className={style.login__form__item__label}
87            >
88              Fullname
89            </label>
90            <input
91              type="text"
92              id="Fullname"
93              name="Fullname"
94              placeholder="Fullname"
95              className={style.login__form__item__input}
96            />
97          </div> */}
```

Sehingga hasilnya seperti berikut :



- Buka file index.tsx pada folder views/auth/login dan modifikasi codenya seperti berikut (Untuk line 64 sampai kebawah tidak ada perubahan)

```
1  import Link from "next/link";
2  import style from "../../auth/login/login.module.scss";
3  import { useState } from "react";
4  import { useRouter } from "next/router";
5  import { signIn } from "next-auth/react";
6
7  const TampilanLogin = () => {
8    const [isLoading, setIsLoading] = useState(false);
9    const { push, query } = useRouter();
10
11   const callbackUrl: any = query.callbackUrl || "/";
12   const [error, setError] = useState("");
13
14   const handleSubmit = async (event: any) => {
15     event.preventDefault();
16     setError("");
17     setIsLoading(true);
18
19     // event.preventDefault();
20     // const form = event.currentTarget;
21     // const formData = new FormData(event.currentTarget);
22     // const email = formData.get("email") as string;
23     // const fullname = formData.get("Fullname") as string;
24     // const password = formData.get("Password") as string;
25
26     try {
27         const res = await signIn("credentials", {
28             redirect: false,
29             email: event.target.email.value,
30             fullname: event.target.Fullname.value,
31             password: event.target.Password.value,
32             callbackUrl,
33     });
34     // validasi email dan password
35     if (!res?.error) {
36       setIsLoading(false);
37       push(callbackUrl);
38     } else {
39         setIsLoading(false);
40         setError(res?.error || "Login failed");
41     }
42     } catch (error) {
43         setIsLoading(false);
44         setError("wrong email or password");
45     }
46 };
```

Note pastikan tulisan password pada event.password.value pada line 48 sama dengan yang ada di

```
1  <input
2                 type="password"
3                 id="password"
4                 name="password"
5                 placeholder="password"
6                 className={style.login__form__item__input}
7               />
8             </div>
```

- Buka file servicefirebase.ts dan tambahkan code di line 25-38

```
my-app-16 > src > utils > db > servicefirebase.ts > retrieveDataByID
Windsurf: Refactor | Explain | Generate JSDoc | ×
26  export async function retrieveDataByID(collectionName: string, id: string) {
27  }
28  // export async function retrieveDataByID(collectionName: string, id: string) {
29  //   const snapshot = await getDoc(doc(db, collectionName, id));
30  //   const data = snapshot.data();
31  //   return data;
32  // }
Windsurf: Refactor | Explain | ×
33  export async function signIn(
34    email: string
35  ) {
36    const q = query(collection(db, "users"), where("email", "==", email));
37    const querySnapshot = await getDocs(q);
38    const data = querySnapshot.docs.map((doc) => ({
39      id: doc.id,
40      ...doc.data(),
41    }));
42    if (data) {
43      return data[0];
44    } else {
45      return null;
46    }
47  }
```

BAGIAN 3 – Authorize di NextAuth (Database Login)

- Buka file [...nextauth].ts modifikasi menjadi berikut (pada bagian providers)

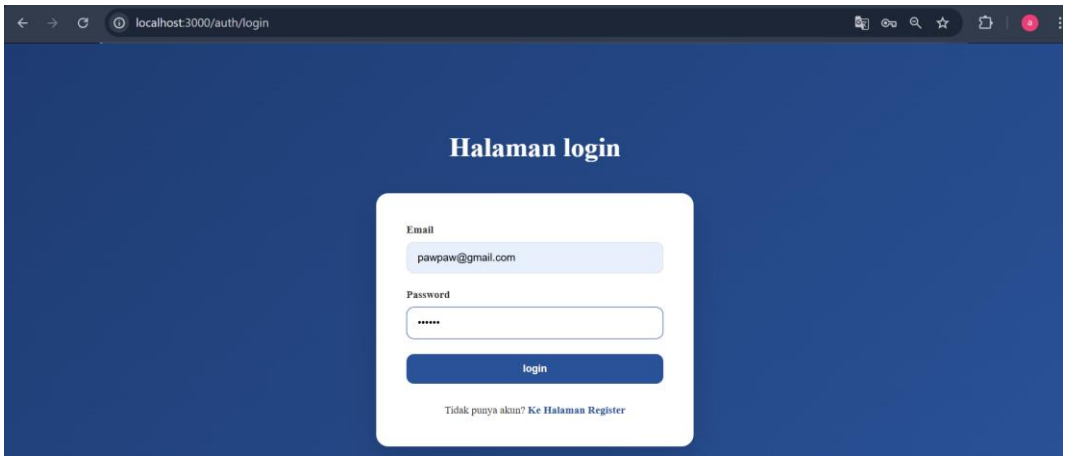
```
import { signIn } from "@/utils/db/servicefirebase";
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcrypt";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,

  providers: [
    // credentials provider untuk login
    CredentialsProvider({
      name: "credentials",
      credentials: {
        // fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials.password)  return null;

        const user: any = await signIn(credentials.email);
        if (user) {
          const isPasswordValid = await bcrypt.compare(
            credentials.password,
            user.password,
          );
          if (isPasswordValid) {
            // pastikan mmengembalikan object user yang bersih
            return {
              id: user.id,
              email: user.email,
              fullname: user.fullname,
              role: user.role,
            };
          }
        }
        return null;
      },
    }),
  ],
```

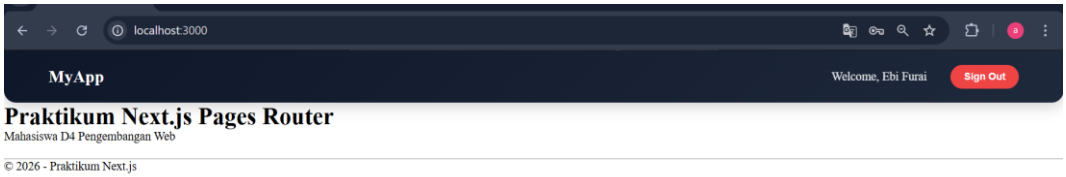
BAGIAN 4 – Tambahkan Role ke Token

- JWT Callback pada file [...nextauth].ts Modifikasi menjadi

```
callbacks: {
    // async redirect({ url, baseUrl }) {
    //   if (url.startsWith(baseUrl)) return url;
    //   if (url.startsWith("/")) return `${baseUrl}${url}`;
    //   return `${baseUrl}/profile`;
    // },

    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
        token.role = user.role;
      }
      return token;
    },
    async session({ session, token }: any) {
      if (token.email) {
        session.user.email = token.email;
      }
      if (token.fullname) {
        session.user.fullname = token.fullname;
      }
      if (token.role) {
        session.user.role = token.role;
      }
      return session;
    },
  },
```

- Jalankan browser http://localhost:3000/auth/login

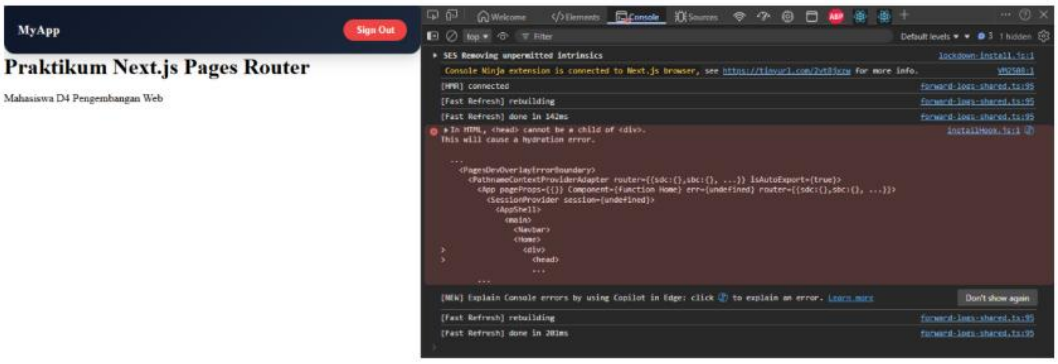




Note ERROR :

JIKA TERDAPAT ERROR SEPERTI INI DISEBABKAN KARENA

In this case, the problem is that a <head> tag is being rendered inside a , which is invalid HTML. The <head> element must be a direct child of <html>, not nested inside other elements.

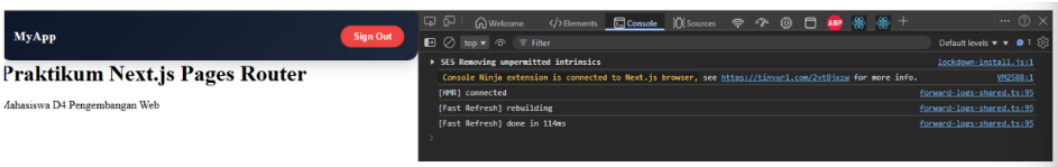


SOLUSINYA BUKA FILE SRC/VIEWS/AUTH/LOGIN/INDEX.TSX

TAMBAHKAN <> </> SEPERTI PADA GAMBAR BERIKUT (LINE 67 DAN 116)

15.praktikum-Login_multi_role > my-app > src > views > auth > login > index.tsx > ...

```
1   import Link from "next/link";
2   import style from "../../auth/login/login.module.scss";
3   import { useState } from "react";
4   import { useRouter } from "next/router";
5   import { signIn } from "next-auth/react";
6
    Windsurf: Refactor | Explain | Generate JSDoc | X
7   const Tampilanlogin = () => {
8     const [isLoading, setIsLoading] = useState(false);
9     const { push, query } = useRouter();
10
11    const callbackUrl: any = query.callbackUrl || "/";
12    const [error, setError] = useState("");
      Windsurf: Refactor | Explain | Generate JSDoc | X
13  >   const handleSubmit = async (event: any) => {···
64    };
65
66    return (
67      <>
68  >     <div className={style.login}>···
115       </div>
116     </>
117   );
118 };
119
120 export default Tampilanlogin;
121
```



BAGIAN 5 – Callback URL Logic

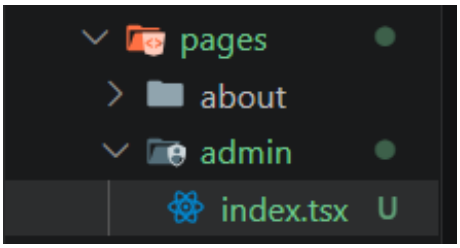
- Modifikasi withAuth.ts pada folder src/middleware

```
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const Url = new URL("/auth/login", req.url);
        Url.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(Url);
        // perlindungan untuk halaman yang membutuhkan autentikasi, jika tidak ada token maka akan diarahkan ke halaman login menggunakan middleware
      }
    }
    return middleware(req, next);
  }
}
```

Tujuannya:
Setelah login, user kembali ke halaman sebelumnya.

BAGIAN 6 – Membuat halaman Admin dan authoriz

- Buat halaman admin



- Pada index.tsx tambahkan code berikut

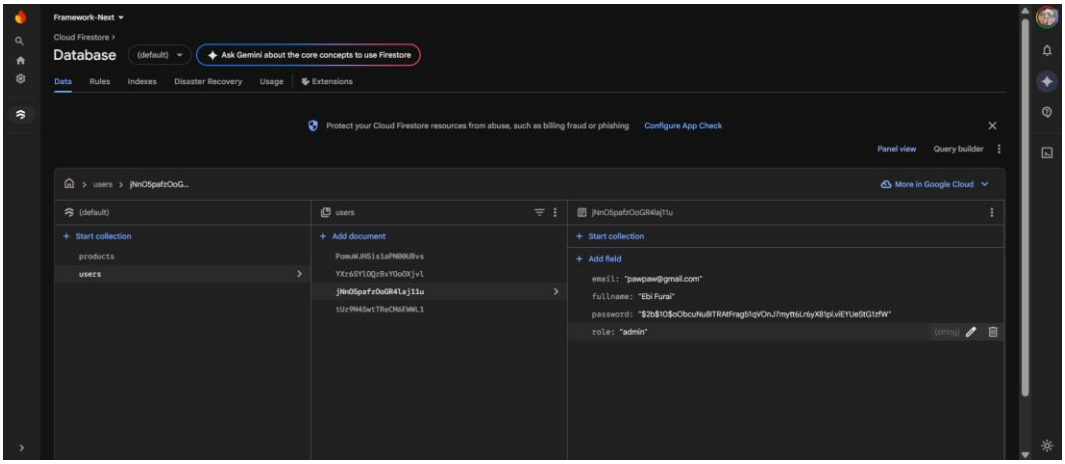
```
const HalamanAdmin = () => {
  return (
    <div>
      <div className="admin">
        <h1>Halaman Admin</h1>
        <p>
          Selamat datang di halaman admin! Anda memiliki akses penuh ke semua
          fitur dan data di aplikasi ini. Di sini, Anda dapat mengelola
          pengguna, melihat laporan, dan melakukan tugas administratif lainnya.
          Pastikan untuk menggunakan hak akses Anda dengan bijak dan menjaga
          keamanan data pengguna.
        </p>
      </div>
    </div>
  );
};

export default HalamanAdmin;

```

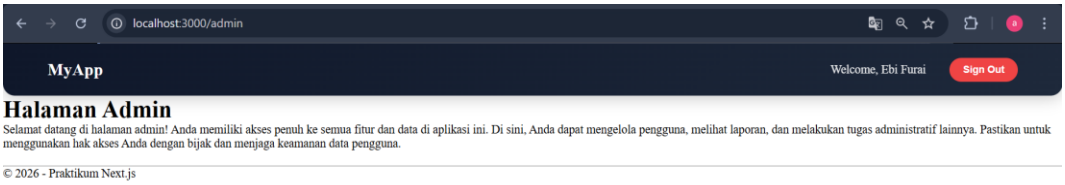
- Modifikasi withAuth.ts

```
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

const hanyaAdmin = ["/admin"];
export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const Url = new URL("/auth/login", req.url);
        Url.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(Url);
        // perlindungan untuk halaman yang membutuhkan autentikasi, jika tidak ada token maka akan diarahkan ke halaman login menggunakan middleware
      }
      if (token.role !=="admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
    }
  }
    return middleware(req, next);
  };
}

```

- Jalankan browser localhost:3000/produk dan pada status sudah login. Rubah urlnya menjadi http://localhost:3000/admin maka user akan diarahkan ke localhost. Pada intinya role selain admin tidak bisa mengakses

- Untuk mencoba halaman admin rubah role pada firebas pada salah satu akun dan jalankan http://localhost:3000/admin



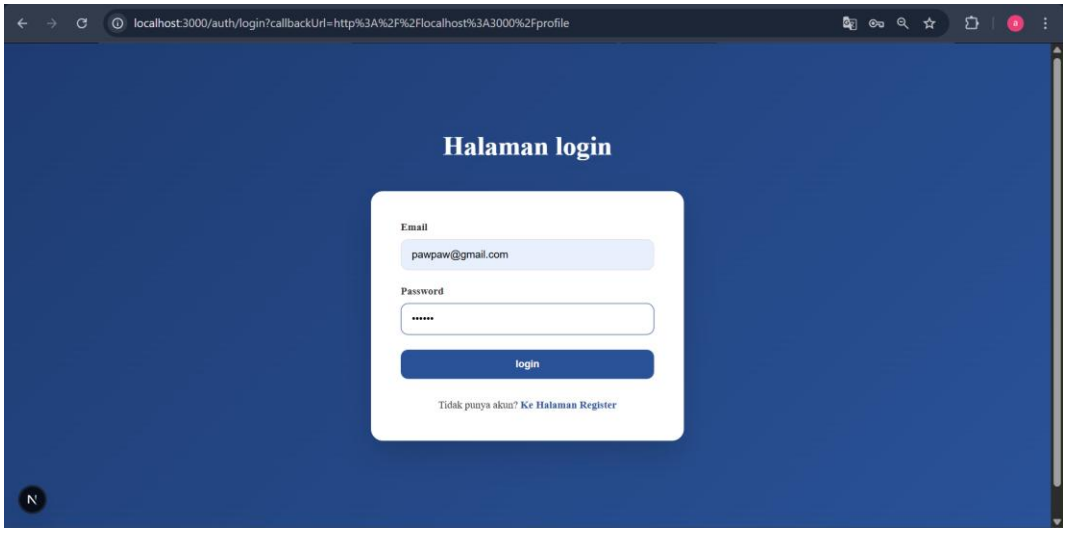


D. Pengujian

Uji 1 – Login Valid

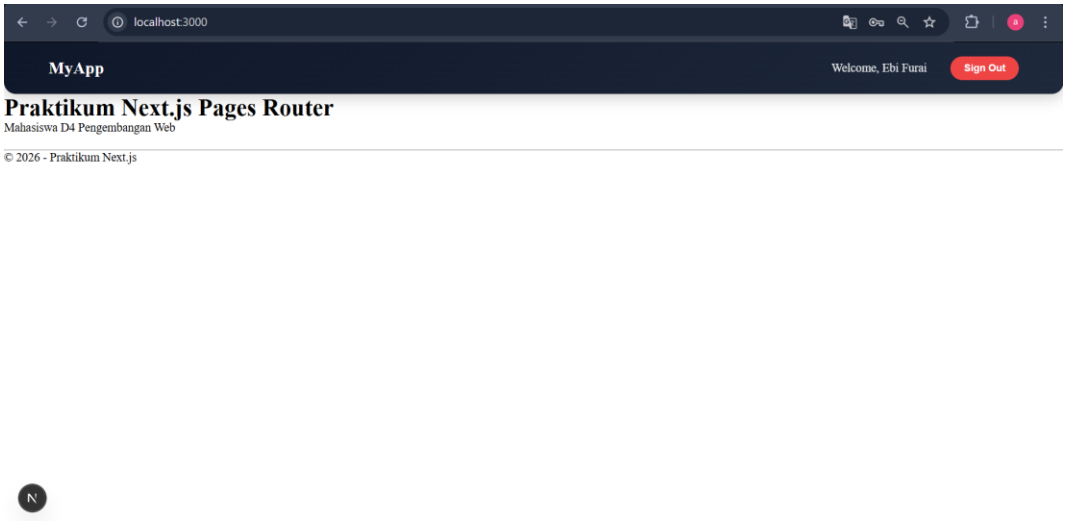
Input:

- Email benar
- Password benar



Hasil:

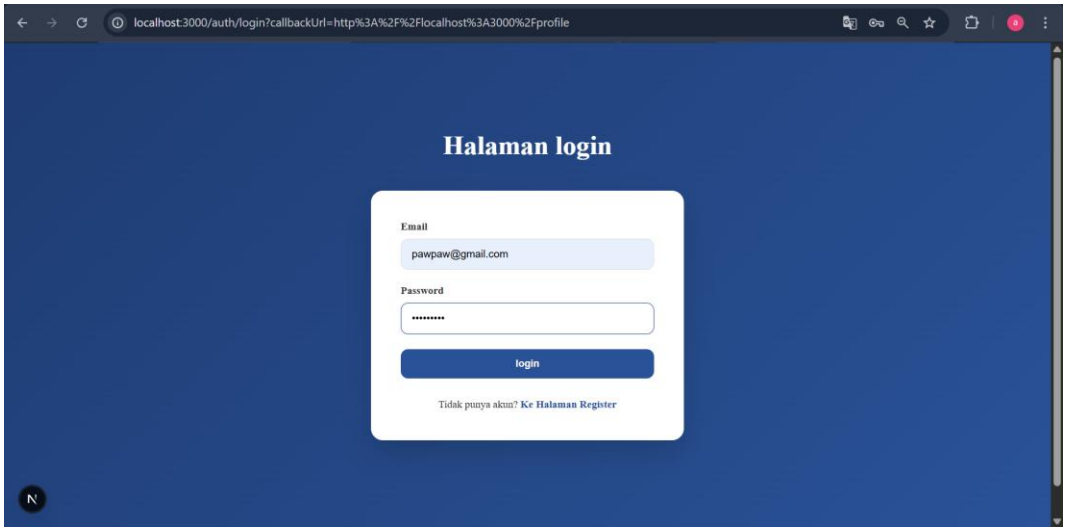
- Login berhasil
- Redirect sesuai callbackUrl



Uji 2 – Password Salah

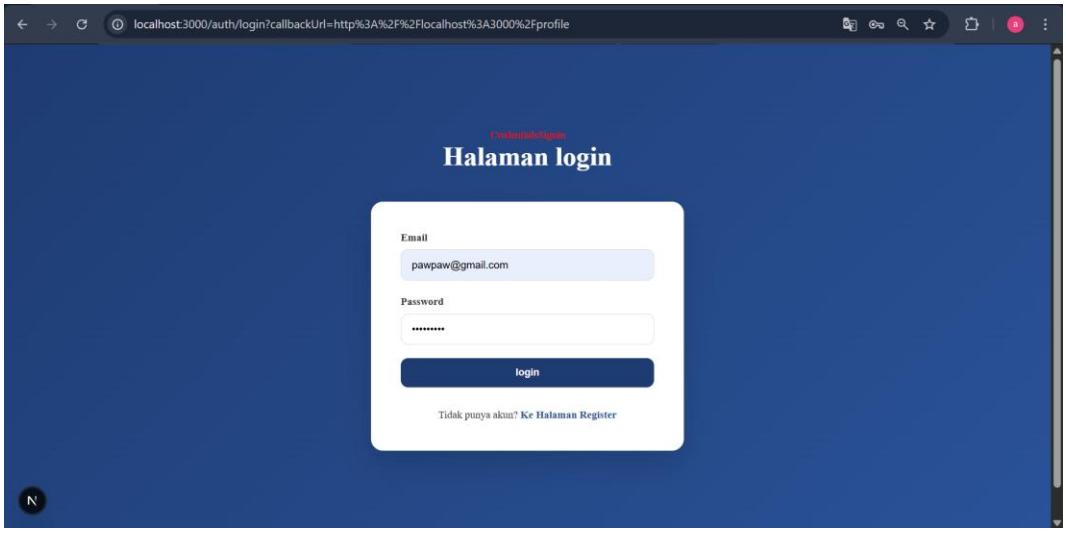
Input:

- Email benar
- Password salah



Hasil:

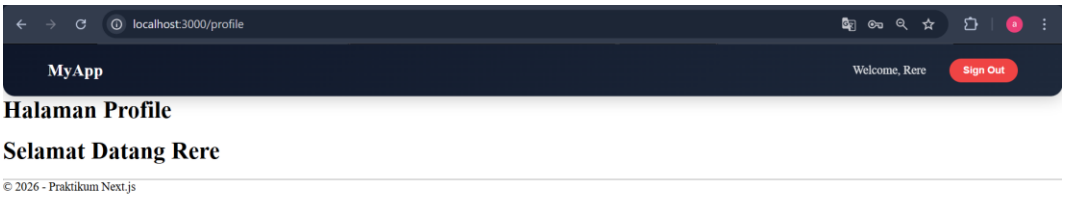
- Error message tampil
- Tidak login



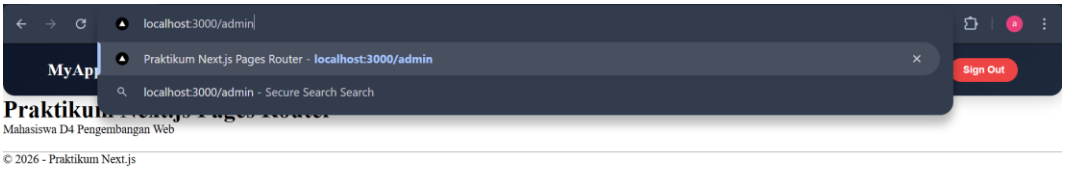
Uji 3 – Akses Admin sebagai User

Login sebagai:

- role: user

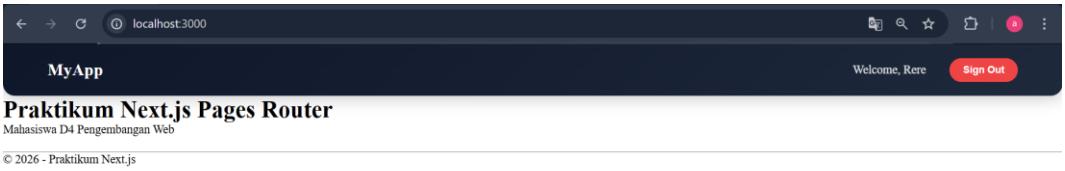


Akses:
/admin



Hasil:

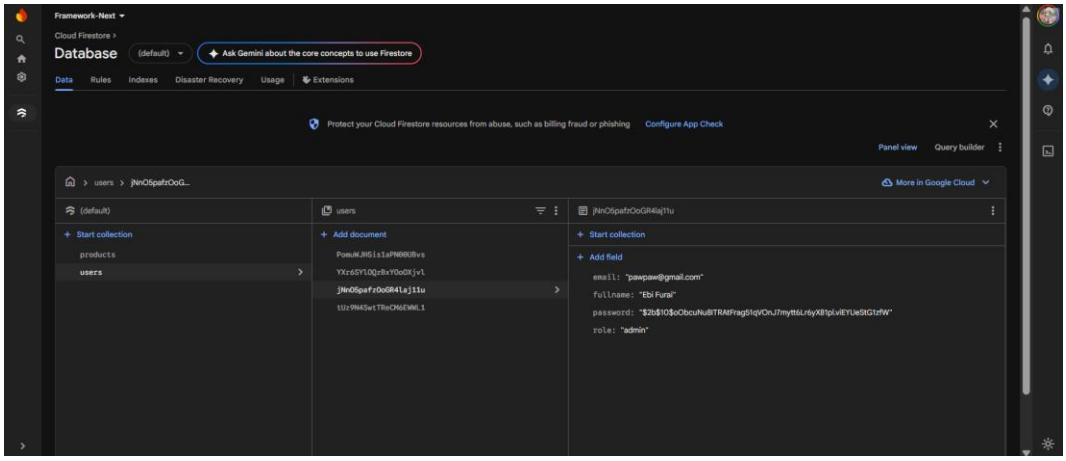
- Redirect ke home

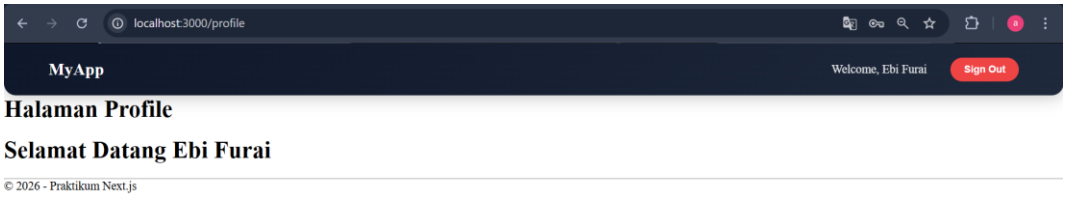


Uji 4 – Akses Admin sebagai Admin

Login sebagai:

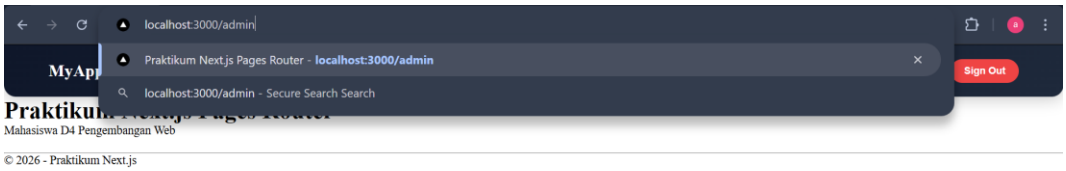
- role: admin





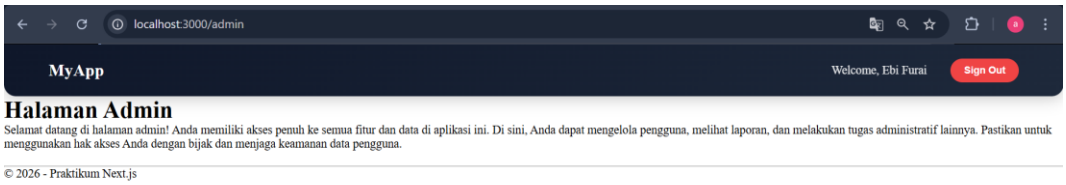
Akses:

/admin



Hasil:

- Bisa masuk halaman admin



E. Struktur Database Users

Collection: users

Field	Tipe
email	string
password	string (hashed)
role	string
fullName	string

F. Perbandingan Sistem

Fitur	Sebelum	Sekarang
Login	Hardcoded	Database
Password	Plaintext	Hashed
Role	Tidak ada	Ada
Redirect	Manual	Callback URL
Middleware	Basic	Role-based

G. Tugas Praktikum

1. Implementasikan login database.
   Src/pages/api/auth/ [...nextauth].ts

```
1  async authorize(credentials) {
2    if (!credentials?.email || !credentials.password)  return null;
3
4    const user: any = await signIn(credentials.email);
5    if (user) {
6      const isPasswordValid = await bcrypt.compare(
7        credentials.password,
8        user.password,
9      );
10     if (isPasswordValid) {
11       // pastikan mmengembalikan object user yang bersih
12       return {
13         id: user.id,
14         email: user.email,
15         fullname: user.fullname,
16         role: user.role,
17       };
18     }
19   }
20   return null;
21 },
```

Src/utils/db/servicefirebase.ts

```
1  export async function signIn(
2    email: string
3  ) {
4    const q = query(collection(db, "users"), where("email", "==", email));
5    const querySnapshot = await getDocs(q);
6    const data = querySnapshot.docs.map((doc) => ({
7      id: doc.id,
8      ...doc.data(),
9    }));
10   if (data) {
11     return data[0];
12   } else {
13     return null;
14   }
15 }
16
```

2. Tambahkan role pada user.
   Src/utils/db/servicefirebase.ts

```
userData.role = "user";
await addDoc(collection(db, "users"), userData)
```

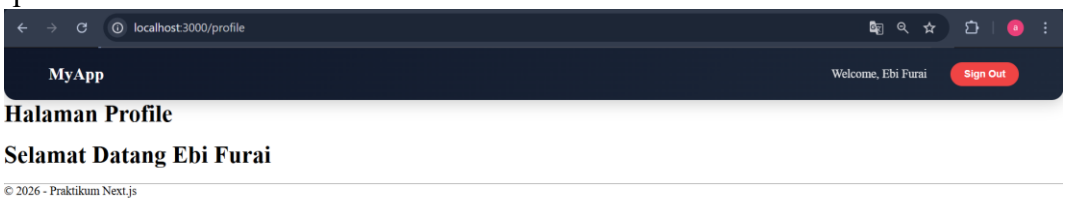
Src/pages/api/auth/ [...nextauth].ts

```
async jwt({ token, account, profile, user }: any) {
  if (account?.provider === "credentials" && user) {
    token.email = user.email;
    token.fullname = user.fullname;
    token.role = user.role;
  }
  return token;
},
```

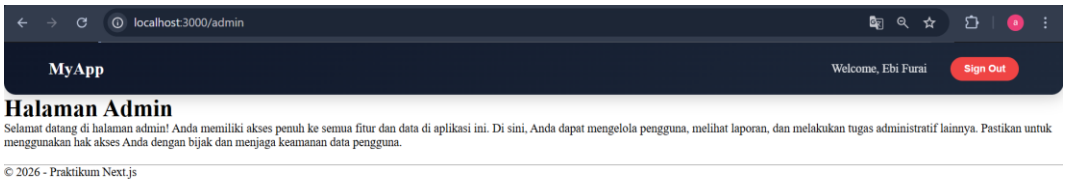
```
if (token.role) {
  session.user.role = token.role;
}
```

3. Buat halaman:

- /profile



- /admin



4. Proteksi /admin hanya untuk admin.
   Src/Middleware/withAuth.ts

```ts
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

const hanyaAdmin = ["/admin"];
export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const Url = new URL("/auth/login", req.url);
        Url.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(Url);
        // perlindungan untuk halaman yang membutuhkan autentikasi, jika tidak ada token maka akan diarahkan ke halaman login menggunakan middleware
      }
      if (token.role !=="admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
    }
  }
    return middleware(req, next);
  };
}
```

   Src/ middleware.ts

```ts
export default withAuth(mainMiddleware as NextMiddleware, ["/produk", "/about", "/profile", "/admin"]);

export const config = {
  matcher: ["/produk", "/about", "/profile", "/admin"],
};
```

5. Implementasikan callback URL.

   views/auth/login/index.tsx

```ts
const callbackUrl: any = query.callbackUrl || "/";
```

```
const res = await signIn("credentials", {
            redirect: false,
            email: event.target.email.value,
            password: event.target.password.value,
            callbackUrl,
    });
```

```
if (!res?.error) {
  setIsLoading(false);
  push(callbackUrl);
```

H. Pertanyaan Analisis

1. Mengapa password harus diverifikasi dengan bcrypt.compare?
   - Password yang disimpan di database sudah dalam bentuk hash, sehingga tidak bisa dibandingkan langsung dengan password asli yang diinput user. Oleh karena itu digunakan bcrypt.compare untuk mencocokkan password input dengan hash di database. Hal ini penting untuk menjaga keamanan agar password tidak tersimpan dalam bentuk asli.

2. Mengapa role disimpan di token?
   - Role disimpan di token agar informasi hak akses user bisa digunakan di berbagai bagian aplikasi tanpa perlu mengambil data ulang dari database. Dengan begitu, proses pengecekan akses menjadi lebih cepat dan efisien.

3. Apa fungsi callbackUrl?
   - CallbackUrl digunakan untuk menentukan halaman tujuan setelah user berhasil login. Biasanya user akan dikembalikan ke halaman yang sebelumnya ingin

diakses, sehingga alur penggunaan menjadi lebih nyaman dan tidak harus mengakses ulang secara manual.

4. Mengapa middleware penting untuk security?
   - Middleware berfungsi untuk mengecek autentikasi dan otorisasi sebelum halaman diakses oleh user. Dengan middleware, proses pengecekan dilakukan di server sebelum halaman dirender, sehingga lebih aman dan tidak mudah dimanipulasi dari sisi client.

5. Apa risiko jika role tidak dicek di middleware?
   - Jika role tidak dicek, maka semua user dapat mengakses halaman yang seharusnya terbatas, seperti halaman admin. Hal ini dapat menyebabkan kebocoran data dan penyalahgunaan fitur oleh user yang tidak memiliki hak akses.

I. Output yang Diharapkan

Mahasiswa menghasilkan:

Login terhubung database

Password diverifikasi

Custom login page

Redirect sesuai callback URL

Middleware aktif

Role-based access berjalan

Insight Penting

Level ini sudah masuk kategori:

Production-ready authentication system

Yang sudah mencakup:

• Authentication

• Authorization

• Session management

• Role-based access control